

Agent per a una Variant de Connecta 4

Pràctica 6

Serafí Nebot Ginard
Universitat de les Illes Balears (UIB)
Grau en Enginyeria Informàtica
Algorismes Avançats, Curs 2025–2026

Resum—Aquesta memòria descriu el disseny i la implementació d'una aplicació Java per jugar a una variant de Connecta 4 sobre un tauler de 7×7 . A més de la inserció vertical clàssica, el joc permet eliminar una fitxa pròpia i rotar el tauler 90 graus a l'esquerra o a la dreta, aplicant gravetat després de cada canvi d'orientació. L'aplicació s'ha implementat amb arquitectura Model-Vista-Controlador, interfície gràfica Swing, animacions de caiguda i rotació, modes humà contra humà, humà contra agent i agent contra agent, i reproducció de partides amb historial. S'han desenvolupat tres agents: un agent aleatori, un agent greedy d'una sola capa i un agent minimax amb profunditat limitada, ordenació de moviments i poda alfa-beta. La memòria formalitza les regles implementades, descriu l'espai d'estats i la funció heurística emprada pel minimax, i discuteix els resultats funcionals obtinguts sense introduir dades experimentals no mesurades.

Termes d'índex—Connecta 4, minimax, poda alfa-beta, branch and bound, heurística, MVC, Java Swing, jocs adversarials.

1. Introducció

El Connecta 4 clàssic és un joc adversarial de dos jugadors en què les fitxes es col·loquen en columnes i cauen fins a la primera posició lliure. Guanya el primer jugador que forma quatre fitxes consecutives en línia horitzontal, vertical o diagonal. La pràctica modifica aquest joc en tres aspectes importants: el tauler és de 7×7 , cada jugador pot eliminar una fitxa pròpia i el tauler complet es pot rotar 90 graus a l'esquerra o a la dreta. Després d'una eliminació o una rotació, les fitxes tornen a caure segons la nova orientació del tauler.

Aquestes modificacions fan que el problema sigui més interessant que el Connecta 4 estàndard. Una jugada pot afavorir el jugador que mou, però també pot crear una línia guanyadora per al contrari; fins i tot pot generar quatre en línia per als dos jugadors alhora, cas que l'enunciat defineix com empat. A més, l'existència de rotacions i eliminacions fa que el joc no sigui estrictament progressiu en el mateix sentit que el joc clàssic: el nombre de fitxes pot disminuir i la forma del tauler pot canviar completament amb una sola acció.

L'objectiu obligatori de la pràctica és construir una aplicació amb interfície gràfica que permeti jugar contra una màquina i que aquesta determini la jugada mitjançant Branch and Bound. En aquesta implementació s'ha modelat aquest requisit com un agent `MinimaxAgent` amb poda alfa-beta. En jocs de dos jugadors, la cerca minimax proporciona el marc natural per alternar decisions entre maximitzador i minimitzador [1], mentre que la poda alfa-beta actua com el mecanisme de Branch and Bound: descarta branques que ja no poden millorar la decisió final [2].

Com a millores opcionals s'han incorporat modes agent contra agent, un agent aleatori i un agent greedy. Això permet observar estratègies de complexitat molt diferent: una sense coneixement del joc, una heurística local i una cerca adversarial amb profunditat limitada. La interfície també permet reproduir partides automàtiques amb retard, aturar-les, avançar jugades i retrocedir dins de l'historial.

2. Descripció del Problema

2.1. Tauler i Torns

El tauler té $N = 7$ files i $N = 7$ columnes. Cada cel·la pot estar buida, ocupada pel jugador vermell o ocupada pel jugador groc. Els jugadors alternen torns, i en cada torn el jugador actual tria exactament una acció legal. La implementació fa servir el jugador vermell com a primer jugador.

Les accions possibles són:

- **Inserir:** triar una columna que no estigui plena i col·locar-hi una fitxa pròpia. La fitxa cau fins al fons o fins quedar damunt una altra fitxa.
- **Eliminar:** triar una fitxa pròpia del tauler i retirar-la. Les fitxes que hi havia per damunt cauen per ocupar l'espai.
- **Rotar a l'esquerra:** girar tot el tauler 90 graus en sentit antihorari i aplicar gravetat.
- **Rotar a la dreta:** girar tot el tauler 90 graus en sentit horari i aplicar gravetat.

Un moviment és terminal si, després d'aplicar-lo, hi ha quatre fitxes consecutives d'algun jugador. Si només un

jugador té quatre en línia, aquest jugador guanya, encara que no sigui necessàriament qui ha fet la jugada. Si tots dos jugadors tenen quatre en línia després del mateix moviment, la partida acaba en empat. Si el tauler queda ple i no hi ha cap guanyador, també acaba en empat. Seguint l'enunciat, quan el tauler és ple no es continua jugant a base d'eliminar fitxes.

2.2. Espai de Moviments

En una posició no terminal, el nombre de moviments legals pot ser molt superior al Connecta 4 clàssic. En el joc original només hi pot haver fins a 7 moviments, un per columna. En aquesta variant hi pot haver fins a 7 insercions, fins a p eliminacions, on p és el nombre de fitxes pròpies del jugador actual, i dues rotacions. Per tant, el factor de ramificació màxim és

$$b \leq 7 + p + 2. \quad (1)$$

Com que p pot arribar a ser aproximadament la meitat del tauler, la cerca exhaustiva completa és massa costosa per fer-la a cada torn dins d'una interfície interactiva. Per aquest motiu el minimax implementat és de profunditat limitada i utilitza una funció heurística per valorar les fulles no terminals.

3. Fonaments Teòrics

3.1. Model d'Estat

L'estat del joc es representa com

$$S = (B, j, r), \quad (2)$$

on B és el tauler, j és el jugador a qui toca moure i r és el resultat actual: partida en curs, victòria del vermell, victòria del groc o empat. Aquesta separació és la que apareix a la classe `GameState`: el tauler conté la geometria i la gravetat, mentre que l'estat decideix quins moviments són legals i quan s'ha acabat la partida.

La transició d'estat es defineix com

$$S' = \text{apply}(S, m), \quad (3)$$

on m és un moviment legal. Primer es construeix un nou tauler aplicant la física del moviment, després s'avaluen les condicions de victòria i finalment es canvia el torn. La implementació utilitza objectes immutables: aplicar un moviment no modifica l'estat anterior, sinó que retorna un estat nou. Això és especialment important per al minimax, ja que la cerca explora moltes línies hipotètiques i no ha de corrompre la partida real.

3.2. Funció Heurística

La funció heurística de `MinimaxAgent` combina quatre components:

$$h(S) = h_{\text{terminal}}(S) + h_{\text{finestres}}(S) + h_{\text{centre}}(S) + h_{\text{material}}(S). \quad (4)$$

El component terminal domina tots els altres. Una victòria del maximitzador rep un valor molt alt i una derrota rep un valor molt baix. També s'afegeix un petit ajust segons la profunditat restant: una victòria trobada abans és preferible a una victòria més tardana, i una derrota tardana és menys dolenta que una derrota immediata.

El component principal és $h_{\text{finestres}}$. Una finestra és qualsevol segment de quatre cel·les consecutives en una de les quatre direccions guanyadores: horitzontal, vertical, diagonal principal o diagonal secundària. Cada finestra es classifica segons quantes fitxes pròpies i contràries conté. Si conté fitxes dels dos jugadors, queda bloquejada i val 0, perquè cap jugador pot completar exactament aquella línia. Si només conté fitxes d'un jugador i buits, es valora segons la proximitat a una línia de quatre.

Taula 1. PESOS EMPRATS PER VALORAR FINESTRES DE QUATRE CEL·LES.

Situació	Punts propis	Punts oponent
Quatre fitxes	+100000	-100000
Tres fitxes	+900	-1100
Dues fitxes	+90	-100
Una fitxa	+8	-8
Bloquejada	0	0

La penalització d'una triple amenaça de l'oponent és una mica més forta que la recompensa d'una triple pròpia. Aquesta decisió fa que l'agent prefereixi bloquejar perills immediats abans de crear amenaces simètriques que podrien arribar tard. El control del centre afegeix pes a les fitxes situades a la columna central, perquè participen en més línies possibles que les columnes extremes. Finalment, el material és un terme petit que suma les fitxes pròpies i resta les contràries; no decideix la partida per si sol, però serveix com a desempat quan dues posicions tenen amenaces similars.

3.3. Minimax

El minimax és l'algorisme clàssic per a jocs deterministes de suma zero amb dos jugadors i informació perfecta [1]. Des del punt de vista del jugador que mou, la seva decisió intenta maximitzar el valor de la posició, mentre que el contrari intenta minimitzar-lo. Per a una profunditat restant d , la recurrència és

$$\begin{aligned} V(S, d) &= h(S) && \text{si } S \text{ es fulla} \\ V(S, d) &= \max_m V(S_m, d-1) && \text{si } j = j_{\text{max}} \\ V(S, d) &= \min_m V(S_m, d-1) && \text{altrament} \end{aligned} \quad (5)$$

on m recorre $M(S)$, $S_m = \text{apply}(S, m)$ i una fulla és un estat terminal o un estat amb $d = 0$. El valor retornat per la cerca no és una probabilitat, sinó una utilitat relativa: valors positius afavoreixen el jugador maximitzador i valors negatius afavoreixen l'oponent.

3.4. Poda Alfa-Beta com a Branch and Bound

La poda alfa-beta manté dos límits durant la cerca: α , el millor valor que el maximitzador ja pot garantir, i β , el millor valor que el minimitzador ja pot garantir [2]. Quan es compleix $\alpha \geq \beta$, la branca actual no pot canviar la decisió presa en un nivell superior i es pot descartar. Aquesta és la part de Branch and Bound de la solució: s'explora l'arbre de joc, però es tallen branques que queden fora dels límits útils.

L'ordenació de moviments no canvia el resultat del minimax, però sí que afecta molt el nombre de nodes. Si els moviments bons es proven primer, els límits α i β es tanquen abans i la cerca evita més successors. Per això `MinimaxAgent` ordena primer moviments amb victòria immediata i després prefereix insercions centrals.

4. Disseny i Implementació

4.1. Arquitectura MVC

L'aplicació segueix el patró Model-Vista-Controlador. La responsabilitat de cada capa és clara:

Taula 2. RESPONSABILITATS PRINCIPALS DE CADA CAPA.

Capa	Responsabilitat
Model	Regles, tauler, moviments, resultat i agents
Vista	Swing, dibuix del tauler, controls i animacions
Controlador	Torns, modes, historial i connexió vista-model

El model no depèn de Swing. Això permet que els agents utilitzin exactament les mateixes regles que la interfície i facilita futurs canvis. La vista tampoc modifica directament el joc: quan l'usuari clica el tauler o un botó, genera un event cap al controlador. El controlador decideix si el moviment és legal, aplica el model i demana a la vista que mostri la transició.

4.2. Decisions de Disseny

La primera decisió rellevant és la immutabilitat del model. Encara que copiar un tauler de 7×7 té un cost, aquest cost és petit i simplifica molt la cerca. Cada branca del minimax pot aplicar moviments sense haver de desfer-los després. Una alternativa seria tenir mètodes `doMove` i `undoMove`, però això faria més fàcil introduir errors subtils amb rotacions i eliminacions.

La segona decisió és no introduir una condició d'empat per repetició d'estat. La variant del joc pot generar cicles, però l'enunciat només defineix dues condicions d'empat:

victòria simultània i tauler ple sense guanyador. Per tant, la implementació respecta estrictament aquestes condicions. Això implica que, en una partida real, dos jugadors podrien repetir posicions; la interfície no ho talla automàticament perquè no forma part de les regles demanades.

4.3. Estructura de Paquets i Classes

El projecte té un paquet base `com.serafinebot.p6` i subpaquets per a model, controlador i vista. Aquesta organització evita dependències circulars i fa que les responsabilitats es puguin localitzar ràpidament. La Taula 3 resumeix les classes principals.

Taula 3. CLASSES PRINCIPALS DE LA PRÀCTICA.

Classe	Funció
Main	Punt d'entrada i arrencada Swing
Board	Tauler, gravetat i rotacions
GameState	Estat, torns i final de partida
Move	Representació d'una acció
GameResult	Resultat de la partida
Agent	Contracte comú dels agents
RandomAgent	Selecció aleatòria
GreedyAgent	Heurística local
MinimaxAgent	Cerca minimax amb poda
GameController	Orquestració MVC
GameFrame	Finestra principal
BoardPanel	Dibuix i animacions

Les classes del model són deliberadament petites. `Player` només identifica els dos jugadors i ofereix el mètode `opponent()`, que s'utilitza constantment en el canvi de torn i en les heurístiques. `Cell` representa el contingut d'una casella i inclou la conversió entre jugador i fitxa. Aquesta separació evita haver d'emprar valors nuls dins del tauler: una cel·la buida és `Cell.EMPTY`, no una absència d'objecte. `MoveType` enumera els quatre tipus de moviment possibles i `Move` guarda les coordenades només quan són necessàries.

Els enums `MatchType` i `AgentType` pertanyen a la vista perquè són opcions de configuració de la interfície, no regles del joc. `MatchType` distingeix humà contra humà, humà contra agent i agent contra agent; en canvi, `AgentType` només indica quin agent concret s'ha de crear. Aquesta decisió substitueix una enumeració combinatòria de modes per una configuració més flexible: si s'afegeix un nou agent, no cal crear totes les combinacions possibles de modes, només afegir un valor nou a `AgentType` i una branca de creació al controlador.

4.4. Flux Complet d'una Jugada

El flux d'una jugada és el mateix per a humans i agents, amb l'única diferència de qui proposa el moviment. En una jugada humana, `BoardPanel` rep el clic del ratolí i calcula la fila i columna dins del tauler. Si la cel·la conté una fitxa del jugador actual, envia `onRemove`; en cas contrari envia `onDrop`. Els botons laterals de rotació

envien `onRotateLeft` o `onRotateRight`. En cap cas la vista modifica el model.

El controlador rep l'event i primer comprova que la partida no estigui ocupada amb una animació o una decisió d'agent. Després comprova si el torn és realment humà. Aquesta comprovació depèn del tipus de partida: en humà contra humà tots dos jugadors són humans; en humà contra agent només ho és el vermell; en agent contra agent no hi ha torns humans. Si el moviment és acceptable, el controlador crida `state.apply(move)`.

El mètode `apply` no modifica l'estat existent. Construeix un tauler nou, avalua el resultat i retorna un nou `GameState`. El controlador desa aquest estat dins de l'historial, actualitza el cursor i demana a la vista que animi el moviment. Mentre dura l'animació, la bandera `busy` impedeix nous clics. En acabar, es tornen a habilitar els controls corresponents al nou torn. Aquest detall és important perquè, si es permetessin entrades durant una animació, la vista podria mostrar una transició que ja no correspon a l'estat real.

En una jugada d'agent, el procés és semblant però el moviment es calcula en segon pla. `GameController` crea un `SwingWorker`, li passa una còpia de l'estat actual i espera el moviment retornat per l'agent. Quan el treball acaba, el moviment entra pel mateix camí que una jugada humana: s'aplica al model, s'afegeix a l'historial i s'anima. Això evita duplicar regles entre jugadors humans i automàtics.

4.5. Model

El model es divideix en tres blocs: representació física del tauler, estat complet de partida i tipus auxiliars. Les subseccions següents descriuen aquests blocs amb més detall; aquí només cal remarcar que totes les operacions de canvi d'estat retornen objectes nous. Aquesta immutabilitat és la propietat que permet reutilitzar el mateix model tant per a la partida real com per a les simulacions dels agents.

4.5.1. Classe Board. `Board` és la classe que conté més lògica física. El tauler és una matriu `Cell[][]` de mida fixa, perquè l'enunciat fixa sempre 7×7 . El mètode `drop` comprova primer que la columna no estigui plena i després recorre la columna des de baix cap amunt fins trobar la primera cel·la buida. Això imita directament la caiguda d'una fitxa sense necessitat de simular frames intermedis al model.

El mètode `remove` només accepta posicions ocupades per una fitxa del jugador actual. Després de buidar la cel·la, crida `applyGravity`. Aquesta funció compacta cada columna amb un índex `writeRow`: recorre la columna de baix cap amunt i mou cada fitxa trobada a la posició lliure més baixa. En acabar, totes les posicions que queden per damunt s'omplen amb `EMPTY`. Aquesta tècnica és simple, no depèn de casos especials i garanteix que no quedin forats entre fitxes.

Les rotacions es fan en dues fases. Primer es construeix una matriu nova amb la transformació geomètrica corresponent. Per girar a l'esquerra, la posició (r, c) passa a $(N - 1 - c, r)$; per girar a la dreta passa a $(c, N - 1 - r)$.

Després es torna a aplicar gravetat, perquè una rotació pot deixar fitxes suspeses segons la nova orientació. Aquesta separació entre rotació geomètrica i gravetat fa que la implementació sigui fàcil de verificar.

La detecció de victòria també es concentra a `Board`. El mètode `hasConnectFour` recorre totes les cel·les i, per cada fitxa del jugador buscat, prova quatre direccions: dreta, avall, diagonal descendent i diagonal ascendent. No cal provar les direccions contràries perquè trobarien les mateixes línies duplicades. El cost és proporcional al nombre de cel·les multiplicat per quatre direccions constants.

La correcció d'aquesta classe es pot expressar com una invariant de gravetat: després de qualsevol operació pública, no hi ha cap fitxa amb una cel·la buida immediatament davall dins de la mateixa columna. `drop` la preserva perquè col·loca la fitxa directament a la posició lliure més baixa; `remove` i les rotacions la restauren cridant `applyGravity`. Amb $N = 7$, tant la gravetat com la detecció de quatre en línia tenen cost $O(N^2)$ i són constants a efectes pràctics.

4.5.2. Classe GameState. `GameState` és un record immutable amb tres camps: `board`, `currentPlayer` i `result`. La seva responsabilitat principal és connectar les regles físiques del tauler amb la lògica de torns. Per això `Board` sap eliminar una fitxa d'un jugador concret, però és `GameState` qui decideix que el jugador concret és el jugador a qui toca moure.

El mètode `legalMoves` genera els moviments en un ordre estable: primer les insercions, després les eliminacions pròpies i finalment les dues rotacions. Aquest ordre no és una regla del joc, però aporta determinisme als agents quan hi ha empats de puntuació. També és important que les eliminacions només s'afegeixen si el tauler no és ple, ja que l'enunciat especifica que amb totes les fitxes col·locades la partida acaba en empat i no es pot continuar eliminant.

El mètode `evaluate` comprova sempre els dos jugadors després de cada moviment. Aquesta decisió és necessària perquè una eliminació o una rotació pot crear una línia per al jugador contrari. Si només es comprovés el jugador que ha mogut, el programa acceptaria partides incorrectes. La comprovació doble també permet detectar correctament l'empat per victòria simultània.

La correcció de `GameState` depèn de l'ordre d'aquesta avaluació: primer es calcula el tauler successor, després es comproven les línies guanyadores dels dos jugadors, després l'empat per tauler ple i només si cap cas terminal apareix la partida continua. La generació de moviments té cost $O(N + p)$, on p és el nombre de fitxes pròpies del jugador actual, perquè cal mirar les columnes disponibles i les fitxes que es podrien eliminar.

4.5.3. Resultats i Tipus Auxiliars. `GameResult` és un record petit però útil: evita haver d'interpretar manualment un `GameState` per saber si hi ha guanyador. En una victòria, el camp `winner` conté el jugador; en empat o partida en curs és nul. Els mètodes estàtics `win`, `draw` i `inProgress` fan que la creació de resultats sigui clara i redueixen errors.

`Move` també és intencionadament simple. S'ha preferit un únic record amb fàbriques estàtiques a una jerarquia de subclasses perquè els moviments són valors petits que es creen massivament durant la cerca minimax. Aquesta representació redueix la quantitat de classes, facilita imprimir moviments a l'estat de la interfície i evita polimorfisme innecessari dins dels bucles de cerca.

4.6. Vista i Interacció

La vista està formada per `GameFrame`, que organitza la finestra, i `BoardPanel`, que dibuixa el tauler. La decisió important és que cap de les dues classes aplica regles de joc: només mostren l'estat rebut i transformen les interaccions de l'usuari en events cap al controlador. Això evita que la lògica de partida quedi duplicada entre `Swing` i el model.

4.6.1. Classe `GameFrame`. `GameFrame` és la finestra principal i implementa la interfície `GameView`. La part superior conté el títol de l'aplicació i una breu descripció de les accions disponibles. La zona central conté el tauler i dos botons laterals de rotació. Aquesta disposició es va triar perquè la rotació és una acció global sobre el tauler i visualment és més natural situar-la als costats que dins del panell de configuració.

El panell lateral conté tres blocs. El primer és la configuració de partida: tipus de partida i agents seleccionats. El segon és la reproducció: reproduir, aturar, avançar i retrocedir. El tercer mostra els jugadors, el torn actual, el nombre de fitxes col·locades i el darrer missatge del controlador. Els selectors d'agent es deshabiliten dinàmicament segons el tipus de partida. En humà contra humà no fan falta; en humà contra agent només s'empra l'agent groc; en agent contra agent s'empren tots dos.

També s'ha implementat un botó personalitzat, `SolidButton`, perquè alguns

Look and Feel de `Swing`, especialment en `macOS`, ignoren parcialment el color de fons dels `JButton`. Aquesta classe pinta manualment un rectangle arrodonit i canvia completament el color quan el botó està deshabilitat. Sense això, alguns botons podien aparèixer amb text blanc sobre fons transparent.

4.6.2. Classe `BoardPanel`. `BoardPanel` és un component `Swing` dibuixat manualment. No utilitza una graella de botons perquè el tauler necessita animacions i un aspecte visual més proper al joc. El mètode `paintComponent` calcula la mida de cada cel·la segons la mida disponible de la finestra, dibuixa el fons blau del tauler i pinta cada fitxa com un cercle. Les fitxes eliminables es marquen amb una creu fosca, de manera que el jugador veu quines peces pròpies pot retirar.

Per detectar clics s'utilitza `mousePressed` en lloc de `mouseClicked`. Aquesta elecció fa que la resposta sigui més immediata: `mouseClicked` només es dispara quan `Swing` reconeix una seqüència completa de pressió i alliberament sense moviment, cosa que pot fallar si l'usuari

mou lleugerament el ratolí. Amb `mousePressed`, el panell respon tan aviat com l'usuari prem dins del tauler.

L'animació de caiguda no duplica la lògica de gravetat. El panell rep l'estat anterior i el posterior, compara la columna afectada i troba la fila on ha aparegut la nova fitxa. A partir d'aquí només interpola la coordenada vertical amb una funció cúbica *ease-out*. Això dona una caiguda ràpida al principi i més suau al final. En les rotacions, el panell guarda temporalment l'estat anterior i el dibuixa amb una `AffineTransform`; quan acaba el temporitzador, torna a mostrar l'estat final normal.

4.7. Controlador i Historial

`GameController` és el punt central de coordinació. Manté l'estat actual, els agents seleccionats, el tipus de partida i un historial de posicions. També manté un cursor dins d'aquest historial per implementar els botons *Següent* i *Anterior*. Si l'usuari retrocedeix i després fa una jugada nova, el controlador elimina el futur antic de l'historial, de la mateixa manera que un editor elimina el camí de refer després d'una modificació.

Les decisions dels agents s'executen dins d'un `SwingWorker`. Aquesta decisió és necessària perquè el minimax pot explorar milers de nodes i, si s'executàs directament al fil d'events de `Swing`, congelaria la interfície. En mode agent contra agent, un `Timer` de `Swing` crida periòdicament el següent moviment amb un retard visible, cosa que permet observar la partida en temps real.

El controlador també centralitza l'activació i desactivació de controls. Aquesta responsabilitat no es deixa a la vista perquè depèn de l'estat lògic del joc. Per exemple, els botons de rotació han d'estar actius en un torn humà, però no mentre un agent està pensant ni durant una animació. De la mateixa manera, el botó *Reproduir* només té sentit en mode agent contra agent i quan la partida no ha acabat. Aquesta política queda concentrada al mètode `refresh`, que actualitza l'estat visual després de cada canvi.

La llista `history` desa tots els `GameState` visitats i la llista `moves` desa el moviment que separa cada parella d'estats consecutius. El cursor indica quina posició de l'historial s'està mostrant. Quan l'usuari prem *Anterior*, el controlador redueix el cursor i mostra directament l'estat anterior. Quan prem *Següent*, si el moviment ja existeix a l'historial, el controlador avança sense animació perquè és navegació històrica; si no existeix, demana un moviment nou a l'agent corresponent. Aquest comportament separa clarament la reproducció d'una partida ja calculada de la generació d'una jugada nova.

5. Agents Implementats

5.1. Agent Aleatori

`RandomAgent` és l'agent base. Obté la llista de moviments legals i en tria un uniformement a l'atzar. No diferencia entre inserir, eliminar o rotar, ni mira si el moviment guanya

o perd. El seu valor és de control: serveix per comprovar que el motor de joc accepta moviments legals encara que no hi hagi cap criteri estratègic darrere la decisió.

Aquest agent també és útil durant el desenvolupament. Com que no té cap lògica estratègica, si una partida aleatòria produeix un moviment il·legal o una excepció, el problema gairebé sempre és al model o al controlador, no a l'agent. A més, permet provar ràpidament les animacions i els modes agent contra agent sense esperar el cost del minimax.

La complexitat temporal d'una decisió és $O(|M(S)|)$ per generar els moviments i $O(1)$ per seleccionar-ne un. La qualitat de joc és baixa però el temps de decisió és pràcticament constant per a l'usuari.

5.2. Agent Greedy

GreedyAgent és un agent d'una sola capa. Per cada moviment legal, aplica el moviment sobre una còpia immutable de l'estat i valora només la posició resultant. Si el moviment produeix una victòria pròpia, rep una puntuació molt alta; si produeix una victòria de l'oponent, rep una puntuació molt baixa; si no és terminal, es calcula una suma simple de material i control del centre.

Aquest agent és més fort que l'aleatori perquè reconeix victòries immediates i prefereix posicions amb més presència pròpia. Tot i això, no modela la resposta de l'oponent. Pot triar una jugada aparentment bona que permeti al contrari guanyar immediatament al torn següent. Per aquest motiu és una bona referència intermèdia entre l'agent aleatori i el minimax.

Si b és el nombre de moviments legals i C el cost de valorar un tauler, el cost és $O(b \cdot C)$. En aquesta implementació C és proporcional a les 49 cel·les del tauler, de manera que el cost pràctic és molt baix.

La funció de puntuació greedy no utilitza finestres de quatre cel·les com el minimax. S'ha mantingut més senzilla expressament perquè el seu paper és ser un intermediari entre l'aleatori i la cerca adversarial. Si el greedy tengués una heurística gairebé igual al minimax, seria més difícil distingir si la millora de rendiment ve de la cerca en profunditat o només de la funció d'avaluació. Amb material i centre, el greedy té criteri però continua essent clarament local.

5.3. Agent Minimax

MinimaxAgent explora l'arbre de joc fins a una profunditat fixa. En cada nivell alterna maximització i minimització segons el jugador que ha de moure. Les posicions terminals obtenen valors extrems i les posicions no terminals al límit de profunditat es valoren amb la funció heurística descrita anteriorment.

La profunditat per defecte és de 3 nivells de moviment. En la terminologia de jocs, un nivell d'aquest tipus s'anomena sovint *ply*: és una jugada individual d'un sol jugador, no una ronda completa de dos torns. Per tant, una profunditat 3 vol dir que l'agent considera la seva jugada, la resposta de l'oponent i una jugada pròpia addicional. Aquesta profunditat és un compromís pràctic: prou gran per detectar moltes

amenaces immediates, però prou petita per respondre en una interfície interactiva. Augmentar-la milloraria la qualitat en alguns casos, però el factor de ramificació de la variant fa que el cost creixi ràpidament.

El pseudocodi següent mostra tant la selecció del moviment inicial com la funció recursiva. Els paràmetres α i β són els límits de poda: quan es creuen, la branca actual no pot canviar la decisió final i s'abandona.

Algorithm 1 Minimax amb poda alfa-beta

```

1: Selecció inicial del moviment
2: millorValor  $\leftarrow -\infty$ , millorMoviment  $\leftarrow$  primer moviment
3:  $\alpha \leftarrow -\infty$ ,  $\beta \leftarrow +\infty$ 
4: for all  $m \in$  moviments ordenats de  $s$  do
5:    $s' \leftarrow \text{apply}(s, m)$ 
6:    $v \leftarrow V(s', d - 1, \alpha, \beta, \text{jugadorMax})$ 
7:   if  $v > \text{millorValor}$  then
8:     millorValor  $\leftarrow v$ , millorMoviment  $\leftarrow m$ 
9:   end if
10:   $\alpha \leftarrow \max(\alpha, \text{millorValor})$ 
11: end for
12: return millorMoviment
13:
14: Funció recursiva  $V(s, d, \alpha, \beta, \text{jugadorMax})$ 
15: if  $d = 0$  o  $s$  és terminal then
16:   return  $h(s)$ 
17: end if
18: maximitza  $\leftarrow$  el jugador de  $s$  és jugadorMax
19: if maximitza then
20:   valor  $\leftarrow -\infty$ 
21: else
22:   valor  $\leftarrow +\infty$ 
23: end if
24: for all  $m \in$  moviments ordenats de  $s$  do
25:    $s_m \leftarrow \text{apply}(s, m)$ 
26:   fill  $\leftarrow V(s_m, d - 1, \alpha, \beta, \text{jugadorMax})$ 
27:   if maximitza then
28:     valor  $\leftarrow \max(\text{valor}, \text{fill})$ 
29:      $\alpha \leftarrow \max(\alpha, \text{valor})$ 
30:   else
31:     valor  $\leftarrow \min(\text{valor}, \text{fill})$ 
32:      $\beta \leftarrow \min(\beta, \text{valor})$ 
33:   end if
34:   if  $\alpha \geq \beta$  then
35:     interrompre el bucle
36:   end if
37: end for
38: return valor

```

Sense poda, el cost temporal és $O(b^d)$, on d és la profunditat [1]. Amb alfa-beta, el pitjor cas continua essent $O(b^d)$, però amb una bona ordenació de moviments el nombre de nodes visitats es redueix molt. En el cas ideal clàssic, la complexitat efectiva s'aproxima a $O(b^{d/2})$ [2]. Per això el programa desa estadístiques de nodes explorats, podes i temps. En aquesta memòria s'utilitzen per justificar el cost

d'una decisió concreta, no com a base d'una comparativa estadística.

5.4. Ordenació de Moviments i Estadístiques

Abans d'explorar successors, `MinimaxAgent` els ordena amb una puntuació ràpida. Aquesta puntuació no és la mateixa que la heurística completa: només serveix per decidir l'ordre de visita. Les victòries immediates es posen al principi, després es prefereixen insercions properes al centre i les altres accions queden amb menor prioritat. L'objectiu és que alfa-beta trobi aviat bons límits i pugui tallar més branques.

L'agent guarda tres mesures de la darrera cerca: nodes visitats, podes realitzades i temps de decisió en mil·lisegons. Aquestes dades es mostren al missatge d'estat quan juga el minimax. No s'han emprat per construir una taula experimental, però sí per comprovar que la profunditat triada manté un temps de resposta adequat per a una interfície interactiva.

5.5. Discussió Qualitativa dels Agents

Els tres agents representen tres nivells d'intel·ligència artificial. L'aleatori no utilitza cap informació del joc i, per tant, és una línia base mínima. El greedy incorpora una noció de posició però només mira un moviment. El minimax incorpora oposició explícita: assumeix que el contrari també intentarà obtenir el millor resultat possible. Aquesta diferència és essencial en jocs adversarials, perquè una jugada no s'ha de valorar només pel que crea ara, sinó també pel que permet respondre.

En aquesta variant concreta, la diferència entre greedy i minimax és especialment visible amb rotacions i eliminacions. Una rotació pot semblar dolenta localment perquè no augmenta el material ni ocupa el centre, però pot preparar o impedir una línia diagonal després de la resposta del contrari. El greedy tendeix a infravalorar aquests moviments; el minimax, en canvi, pot acceptar una jugada amb puntuació local modesta si les respostes futures són favorables.

6. Resultats Funcionals i Discussió

En aquesta pràctica no s'ha fet una campanya experimental amb moltes partides ni s'han recollit dades estadístiques de victòries, temps mitjans o percentatges de poda. Per tant, aquest apartat no presenta una comparativa quantitativa entre agents. Els resultats que es poden defensar són funcionals: el programa implementa les regles exigides, permet jugar amb diferents tipus de participants i mostra el cost de cada decisió minimax quan aquest agent intervé.

La Taula 4 resumeix les comprovacions principals de la implementació. No són dades experimentals comparatives, sinó casos de validació que confirmen que les parts essencials del joc responen segons les regles de l'enunciat.

Taula 4. RESULTATS FUNCIONALS VALIDATS.

Funcionalitat	Resultat comprovat
Inserció	La fitxa cau fins a la cel·la lliure més baixa.
Eliminació	Només es poden llevar fitxes pròpies i s'aplica gravetat.
Rotació	El tauler gira 90 graus i es compacta segons la nova orientació.
Final de partida	Es detecten victòria, empat simultani i tauler ple.
Agents	Els tres agents generen sempre moviments legals.
Historial	Es pot revisar una partida automàtica endavant i enrere.

La discussió dels agents s'ha de llegir en aquest mateix sentit qualitatiu. L'agent aleatori serveix per comprovar que el motor accepta qualsevol moviment legal sense coneixement estratègic. L'agent greedy comprova que una heurística local pot aprofitar oportunitats immediates, però també mostra la limitació de no anticipar la resposta rival. El minimax és l'agent que satisfà el requisit algorítmic central: explora diversos nivells, assigna valors als estats futurs i talla branques amb poda alfa-beta.

Les estadístiques mostrades pel controlador per a `MinimaxAgent` tenen una funció diagnòstica. El nombre de nodes, el nombre de podes i el temps de decisió permeten veure si una jugada concreta ha requerit més cerca que una altra, però no s'han agregat en una taula de rendiment. Per obtenir una comparativa empírica real, caldria executar moltes partides automàtiques per parella d'agents, alternar el color inicial i calcular mitjanes i percentatges. Això queda fora de l'abast implementat aquí.

7. Conclusions

S'ha desenvolupat una aplicació Java completa per jugar a la variant proposada de Connecta 4. El model implementa les regles essencials: inserció amb gravetat, eliminació de fitxes pròpies, rotació del tauler, detecció de quatre en línia, empat simultani i empat per tauler ple. La separació MVC facilita mantenir les regles independents de la interfície i permet reutilitzar el mateix model per humans, agents i futures comparatives.

Pel que fa als agents, s'han implementat tres nivells de complexitat. L'agent aleatori serveix de referència mínima, el greedy introdueix una heurística local i el minimax amb poda alfa-beta satisfà el requisit de cerca adversarial amb Branch and Bound. La funció heurística del minimax combina finestres de quatre cel·les, control del centre i material, amb pesos que prioritzen bloquejar amenaces immediates.

Durant les proves manuals s'ha observat que és molt difícil guanyar contra el minimax, però no impossible. Aquest comportament és coherent amb el fet que l'agent no resol el joc complet: utilitza una profunditat limitada i una heurística aproximada, de manera que pot deixar escapar posicions favorables quan la seqüència crítica queda més enllà de l'horitzó de cerca o no és valorada prou bé.

Com a extensions naturals, es podrien afegir profunditats configurables per al minimax i una comparació explícita entre minimax amb i sense ordenació de moviments.

Referències

- [1] GeeksforGeeks, “Minimax Algorithm in Game Theory | Set 1 (Introduction),” 2026. [Online]. Available: <https://www.geeksforgeeks.org/dsa/minimax-algorithm-in-game-theory-set-1-introduction/>
- [2] GeeksforGeeks, “Minimax Algorithm in Game Theory | Set 4 (Alpha-Beta Pruning),” 2023. [Online]. Available: <https://www.geeksforgeeks.org/dsa/minimax-algorithm-in-game-theory-set-4-alpha-beta-pruning/>